

Making Gradient Descent Work in Practice

Feature Scaling, Convergence, the Learning Rate, and Feature Engineering

Rokan Uddin Faruqui
Professor, Department of CSE

University of Chittagong
rufaruqui@cu.ac.bd

Week 2 · Part 2

From correct to usable

Part 1 gave us

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$$

$$\mathbf{w} := \mathbf{w} - \alpha \nabla_{\mathbf{w}} J, \quad b := b - \alpha \frac{\partial J}{\partial b}$$

That algorithm is correct. Run it on the flats table as written and it will crawl, or diverge. Today is about the difference between an algorithm that is right and one that actually finishes.

Today

1. Why unscaled features cripple descent
2. Feature scaling: three methods
3. Checking for convergence
4. Choosing α systematically
5. Feature engineering
6. Polynomial regression

Feature Scaling

The problem: features on wildly different scales

Take two features of a flat: size $x_1 \in [300, 2000]$ sq ft, and bedrooms $x_2 \in [0, 5]$. A flat at $x_1 = 2000,\ x_2 = 5$ sells for 50 lakh.

Which parameters are plausible?

$w_1 = 50,\ w_2 = 0.1,\ b = 50$ gives

$$50(2000) + 0.1(5) + 50 \approx 100{,}050 \quad \text{far too large}$$

$w_1 = 0.1,\ w_2 = 50,\ b = 50$ gives

$$0.1(2000) + 50(5) + 50 = 500 \quad \text{still too large}$$

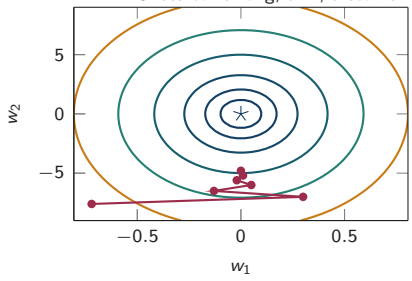
Only a *tiny* w_1 can work, because x_1 is huge.

Key idea

A feature with a **large** range needs a **small** weight, and vice versa. So the cost surface is stretched: J is hypersensitive to w_1 and nearly flat in w_2.

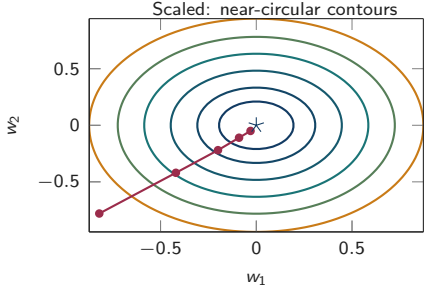
What that does to the contours

Unscaled: a long, thin, tilted valley



Descent oscillates across the valley.

Scaled: near-circular contours



Descent heads almost straight for the minimum.

Key idea

Steepest descent is perpendicular to the local contour. In a thin valley that direction points *across* the valley, not along it, so the path zig-zags and α must stay tiny to avoid overshooting. Round contours remove the problem.

Three ways to scale

Throughout, x_1 ranges over $[300, 2000]$ with mean $\mu_1 = 600$ and standard deviation $\sigma_1 = 450$.

Divide by the max

$$x_{1,\text{scaled}} = \frac{x_1}{\max}$$

$$\frac{300}{2000} \le x_1 \le \frac{2000}{2000}$$

Gives $0.15 \le x_1 \le 1$.

Simplest; range is not centred.

Mean normalization

$$x_{1,\text{scaled}} = \frac{x_1 - \mu_1}{\max - \min}$$

$$\frac{300 - 600}{1700} \le \cdot \le \frac{2000 - 600}{1700}$$

Gives $-0.18 \le x_1 \le 0.82$.

Centred on zero.

Z-score normalization

$$x_{1,\text{scaled}} = \frac{x_1 - \mu_1}{\sigma_1}$$

$$\frac{300 - 600}{450} \le \cdot \le \frac{2000 - 600}{450}$$

Gives $-0.67 \le x_1 \le 3.11$.

The usual default.

Key idea

Aim for roughly $-1 \le x_j \le 1$ for every feature. The bound is a rule of thumb, not a law: $-3 \le x_j \le 3$ is fine, $-0.3 \le x_j \le 0.3$ is fine. Rescale when one feature's range is orders of magnitude away from the others.

Scaling in practice — two rules that matter

Fit on train, apply everywhere

Compute μ_j and σ_j from the *training set only*, store them, and reuse those same numbers for validation, test and production data. Recomputing them on the test set leaks information and quietly inflates your reported score.

Scaling changes what w means

After scaling, w_j is the effect of a *one-standard-deviation* change in feature j, not of one square foot. To quote a coefficient in original units you must unscale it. Predictions are unaffected; interpretations are not.

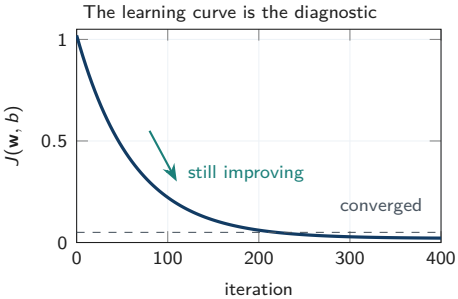
In code

```
mu    = X_train.mean(axis=0)     # per-feature, shape (n,)
sigma = X_train.std(axis=0)
X_train_s = (X_train - mu) / sigma
X_test_s = (X_test - mu) / sigma   # same mu, sigma
```

`axis=0` collapses rows, giving one statistic per feature — the single most common place to get this wrong.

Convergence and the Learning Rate

Checking gradient descent for convergence

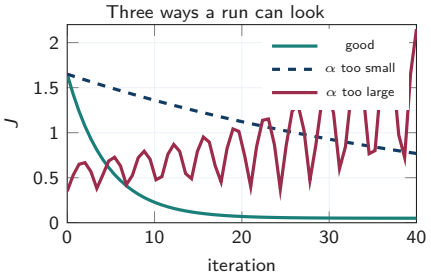


- Plot J against *iteration number*, not against a parameter.
- J must decrease on **every** iteration. One rise means α is too large or the gradient is wrong.
- The curve flattening is convergence.
- The number of iterations needed varies enormously — 30 for one problem, 100,000 for another. There is no way to know in advance; you look.

Automatic convergence test

Declare convergence when J falls by less than ε (say 10^{-3}) in one iteration. Convenient, but choosing ε is itself hard — which is why the plot is preferred.

Choosing the learning rate



The procedure

Try a range of values, each about $3\times$ the last:

$$0.001,\ 0.003,\ 0.01,\ 0.03,\ 0.1,\ 0.3,\ 1$$

Run each for a small, fixed number of iterations and plot J. Pick the *largest* α whose curve still decreases smoothly, then back off one step.

Common pitfall

If J rises with a very small α — say 10^{-6} — the bug is in the gradient, not the learning rate. A correct gradient with a tiny step *must* decrease J.

Feature Engineering and Polynomial Regression

Feature engineering

Predicting the price of a plot of land from its frontage x_1 and depth x_2:

$$f(\mathbf{x}) = w_1 x_1 + w_2 x_2 + b.$$

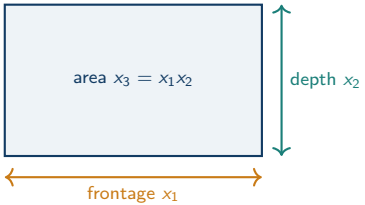
This treats frontage and depth as independent contributions. But what actually drives price is *area*:

$$x_3 = x_1 x_2, \qquad f(\mathbf{x}) = w_1 x_1 + w_2 x_2 + w_3 x_3 + b.$$

The model can now express something no combination of w_1, w_2 could.

Feature engineering

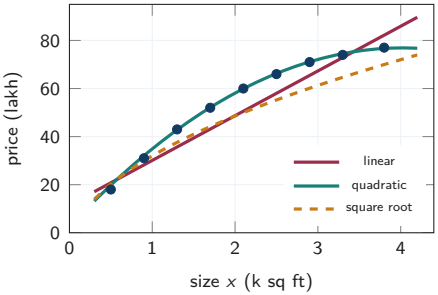
Using domain knowledge to build new features from existing ones — by transforming or combining them — so that the model can represent the relationship you believe is there.



Key idea

The choice of features often matters more than the choice of algorithm. A linear model on well-chosen features routinely beats a complicated model on raw ones.

Polynomial regression



Price rises with size but *flattens*. A straight line cannot bend. Feed the model powers of x as extra features:

$$f(x) = w_1 x + w_2 x^2 + w_3 x^3 + b$$

- Still **linear regression** — linear in the *parameters*. Only the features changed.
- A quadratic eventually turns downward; prices that only flatten are often better served by $\sqrt{x}$, which rises forever but ever more slowly.

Common pitfall

Powers explode the scale: if $x \in [1, 4]$ then $x^3 \in [1, 64]$. **Feature scaling becomes mandatory** once you add polynomial features.

Choosing among candidate feature sets

You now have a way to generate endless models: x, x^2, $\sqrt{x}$, $x_1 x_2$, and any combination. Which do you pick?

What you can do this week

- Plot the data and look at the shape.
- Use domain knowledge — area is a real thing; x^7 is not.
- Compare the fitted w_j: a feature whose weight comes out near zero is not earning its place.

What you cannot do yet

Judging a model by its cost on the *training* data always favours more features. Adding x^{10} can drive J toward zero while making predictions worse. The tools for this — train/validation/test splits, bias and variance, regularization — arrive in Week 3 and Module 2.

Key idea

Treat every engineered feature as a hypothesis about the world, and be ready to discard it.

Wrap-up and Assessment

Summary of Week 2

Concepts

- Features on different scales stretch the cost contours, so descent zig-zags.
- Z-score normalization is the usual fix; fit μ, σ on the training set and reuse them.
- Plot J against iteration. It must decrease, always.
- Choose α by trying $\times 3$ steps and taking the largest that still decreases smoothly.
- New features are hypotheses; polynomial terms are just more features.

Formulas

$$x_{j,\text{scaled}} = \frac{x_j - \mu_j}{\sigma_j}$$

$$f(x) = w_1 x + w_2 x^2 + b \quad \text{(still linear regression)}$$

Next week

Classification: the logistic regression model, decision boundaries, a cost function that squared error cannot provide, and the problem of overfitting.

Descriptive questions

1. Explain why a feature with a large numerical range tends to receive a small weight, and what that does to the shape of the cost contours. **[6]**
2. Define mean normalization and z-score normalization, giving the formula for each. For $x \in [300, 2000]$ with $\mu = 600$, $\sigma = 450$, compute the scaled range under both. **[7]**
3. Why must μ_j and σ_j be computed on the training set alone? Describe concretely what goes wrong if they are recomputed on the test set. **[6]**
4. Describe the learning-curve test for convergence and state the one property the curve must have. Contrast it with an automatic ε-threshold test. **[6]**
5. Give the procedure for choosing α. Explain why the values are spaced by roughly $3\times$ rather than uniformly. **[5]**

Descriptive questions (continued)

6. If J increases while $\alpha = 10^{-6}$, what do you conclude and why? What is your next debugging step? **[5]**
7. Define feature engineering. Give an example, other than area, where a product or ratio of two raw features is more meaningful than either alone. **[5]**
8. "Polynomial regression is not linear regression." Evaluate this claim and justify your verdict precisely. **[5]**
9. Explain why polynomial features make feature scaling mandatory. Illustrate with $x \in [1, 4]$ and the features x, x^2, x^3. **[5]**
10. Why can a model with more features always achieve a lower training cost, and why is that not evidence that it is better? **[5]**

Design questions

1. **Design a scaling pipeline.** Specify a reusable component that scales features for training, validation and production. State what it stores, its interface, and how it behaves when production data contains a value far outside the training range. **[12]**
2. **Design an α search.** Extend the $\times 3$ procedure into an automated routine that returns a recommended α without a human looking at plots. Define the decision rule, what you log, and one failure case your design does not cover. **[12]**
3. **Design the features.** You must predict daily rice-market prices in a Chattogram upazila. Propose six features, at least two of which are engineered from raw measurements, and justify each as a hypothesis about the market. Identify any feature that risks leaking the target. **[12]**
4. **Design a diagnosis procedure.** A colleague reports that training "does not work" on a 40-feature data set. Design an ordered checklist that distinguishes unscaled features, a bad α, and a wrong gradient. Each step must state what you run and what result implicates which cause. **[12]**
5. **Choose a functional form.** Given data where price rises steeply then almost plateaus, compare a quadratic against $\sqrt{x}$ as the engineered feature. Argue from the behaviour of each function outside the training range which you would deploy. **[10]**

Reading

Primary text

G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

→ **§3.3** — extensions of the linear model: interaction terms (feature engineering) and non-linear transformations of the predictors (polynomial regression).

Supplementary

C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

→ **§1.1** — polynomial curve fitting, and the first warning about model complexity that Week 3 takes up as overfitting.

Lab (CSE 816, Week 2): feature scaling and the learning rate; feature engineering and polynomial regression; linear regression with scikit-learn.

Practice quiz: *Gradient descent in practice*.